

Tutorial: Criando um Clone do Gartic Phone em React

Neste tutorial, vamos construir um minijogo interativo baseado no clássico "Telefone Sem Fio" misturado com desenho, no estilo Gartic Phone.

Você aprenderá a manipular o elemento `<canvas>` nativo do HTML5 usando React, gerenciar estados complexos com hooks (`useState`, `useEffect`, `useRef`), criar um cronômetro e implementar um sistema de histórico para a função "Desfazer" (Undo).

O que você vai construir

Vamos criar um jogo com três fases distintas que ocorrem na mesma tela:

1. **Fase de Escrita:** O Jogador 1 digita uma frase e define um tempo.
2. **Fase de Desenho:** O Jogador 2 precisa desenhar a frase antes que o tempo acabe.
3. **Fase de Resultado:** O jogo exibe a frase original e o desenho lado a lado.

Passo 1: Configuração Inicial e Estrutura Básica

Para facilitar, usaremos o **CodePen** com importações via ES Modules, o que dispensa a configuração de um ambiente local com Node.js no início.

O primeiro passo é criar o "esqueleto" do nosso componente e importar as ferramentas do React.

JavaScript

```
import React, { useRef, useState, useEffect } from
"https://esm.sh/react@18";
import { createRoot } from "https://esm.sh/react-dom@18/client";

const GarticClone = () => {
  // O código do nosso jogo vai aqui
  return (
    <div>Olá, Gartic!</div>
  );
};

const root = createRoot(document.getElementById("root"));
root.render(<GarticClone />);
```

Passo 2: A Máquina de Estados (Game State)

Um jogo é basicamente uma máquina que muda de estado. Nosso jogo tem três fases (`write`, `draw`, `result`). Precisamos do hook `useState` para controlar em qual tela o usuário está, além de armazenar as configurações do jogo.

Adicione estes estados dentro do componente principal:

JavaScript

```
// Estados do Jogo
const [gameState, setGameState] = useState('write'); // 'write',
'draw', ou 'result'
const [phrase, setPhrase] = useState('');
const [timeLimit, setTimeLimit] = useState(60);
const [timeLeft, setTimeLeft] = useState(60);
const [finalImage, setFinalImage] = useState(null);
```

Por que usar `useState` aqui? Sempre que esses valores mudarem (por exemplo, quando o `gameState` passar de 'write' para 'draw'), o React saberá que precisa redesenhar a tela inteira para mostrar a nova fase.

Passo 3: Dominando o Canvas no React

O `<canvas>` do HTML5 é uma tela em branco desenhada via JavaScript. No React, não devemos manipular o DOM diretamente. Em vez disso, usamos o hook `useRef` para criar uma "ponte" até o elemento HTML real.

Criando as referências e estados da caneta

JavaScript

```
const canvasRef = useRef(null);
const [isDrawing, setIsDrawing] = useState(false);
const [currentColor, setCurrentColor] = useState('#1e272e');
```

Inicializando o quadro em branco

Usamos o `useEffect` para preparar o quadro assim que o jogador entra na fase de desenho:

JavaScript

```
useEffect(() => {
  if (gameState === 'draw' && canvasRef.current) {
    const canvas = canvasRef.current;
    const context = canvas.getContext('2d');

    // Configurações do pincel para ficar arredondado
    context.lineCap = 'round';
    context.lineJoin = 'round';

    // Pinta o fundo de branco (evita PNGs transparentes no final)
    context.fillStyle = '#ffffff';
    context.fillRect(0, 0, canvas.width, canvas.height);
  }
}, [gameState]); // Executa sempre que o gameState muda
```

As funções de interação do mouse

Para desenhar, precisamos ouvir três eventos do mouse: apertar o botão (`onMouseDown`), mover o mouse (`onMouseMove`) e soltar o botão (`onMouseUp`).

JavaScript

```

const startDrawing = ({ nativeEvent }) => {
  const { offsetX, offsetY } = nativeEvent;
  const context = canvasRef.current.getContext('2d');

  context.strokeStyle = currentColor;
  context.lineWidth = 5;

  context.beginPath();
  context.moveTo(offsetX, offsetY);
  setIsDrawing(true);
};

const draw = ({ nativeEvent }) => {
  if (!isDrawing) return; // Só desenha se o mouse estiver clicado
  const { offsetX, offsetY } = nativeEvent;
  const context = canvasRef.current.getContext('2d');
  context.lineTo(offsetX, offsetY);
  context.stroke(); // Traça a linha
};

const stopDrawing = () => {
  if (isDrawing) {
    const context = canvasRef.current.getContext('2d');
    context.closePath();
    setIsDrawing(false);
  }
};

```

Passo 4: Implementando a Lógica de "Voltar" (Undo)

A funcionalidade de "Desfazer" exige que tiremos "fotos" invisíveis da tela toda vez que o usuário termina um traço.

Para isso, usamos outro `useRef`. **Por que não `useState`?** Porque salvar uma imagem na memória não precisa causar uma atualização visual na interface do React, apenas armazenar o dado em background.

JavaScript

```

const historyRef = useRef([]);

// Chame esta função dentro de stopDrawing()
const saveStateToHistory = () => {
  const canvas = canvasRef.current;
  const context = canvas.getContext('2d');
  // getImageData copia todos os pixels da tela
  historyRef.current.push(context.getImageData(0, 0, canvas.width,
  canvas.height));
};

// Chame esta função no botão de Voltar
const undo = () => {
  if (historyRef.current.length > 1) {
    historyRef.current.pop(); // Descarta o erro
    const previousState = historyRef.current[historyRef.current.length
    - 1];
    const context = canvasRef.current.getContext('2d');
    context.putImageData(previousState, 0, 0); // Restaura a tela
    anterior
  }
};

```

```
}  
};
```

Passo 5: O Cronômetro Automático

Na fase de desenho, o tempo é o maior inimigo. Precisamos de um intervalo que rode a cada 1 segundo (1000ms). Novamente, o `useEffect` entra em ação, monitorando o estado `timeLeft`.

JavaScript

```
useEffect(() => {  
  let timer;  
  // Se estiver na fase de desenho e ainda houver tempo...  
  if (gameState === 'draw' && timeLeft > 0) {  
    timer = setInterval(() => {  
      setTimeLeft((prev) => prev - 1); // Reduz 1 segundo  
    }, 1000);  
  }  
  // Se o tempo acabar...  
  else if (gameState === 'draw' && timeLeft === 0) {  
    finishDrawing(); // Salva e encerra automaticamente  
  }  
  
  // Limpeza de memória crucial no React para evitar vazamentos  
  (memory leaks)  
  return () => clearInterval(timer);  
}, [gameState, timeLeft]);
```

Passo 6: Juntando as Peças (Código Final)

Agora, basta criar o retorno visual (JSX) com base no `gameState` atual usando condicionais (`if`). Abaixo está a arquitetura completa e pronta para uso.

Instruções de Uso: Copie o código abaixo, cole na aba JS de um projeto vazio no CodePen, e você terá o jogo funcionando perfeitamente.

JavaScript

```
import React, { useRef, useState, useEffect } from  
"https://esm.sh/react@18";  
import { createRoot } from "https://esm.sh/react-dom@18/client";  
  
const COLORS = ['#1e272e', '#ff4757', '#2ed573', '#1e90ff', '#ffa502',  
  '#9c88ff'];  
  
const GarticClone = () => {  
  const [gameState, setGameState] = useState('write');  
  const [phrase, setPhrase] = useState('');  
  const [timeLimit, setTimeLimit] = useState(60);  
  const [timeLeft, setTimeLeft] = useState(60);  
  const [finalImage, setFinalImage] = useState(null);  
  
  const canvasRef = useRef(null);  
  const historyRef = useRef([]);  
  const [isDrawing, setIsDrawing] = useState(false);  
  const [currentColor, setCurrentColor] = useState(COLORS[0]);  
  const [isEraser, setIsEraser] = useState(false);
```

```

useEffect(() => {
  let timer;
  if (gameState === 'draw' && timeLeft > 0) {
    timer = setInterval(() => setTimeLeft((prev) => prev - 1),
1000);
  } else if (gameState === 'draw' && timeLeft === 0) {
    finishDrawing();
  }
  return () => clearInterval(timer);
}, [gameState, timeLeft]);

useEffect(() => {
  if (gameState === 'draw' && canvasRef.current) {
    const canvas = canvasRef.current;
    const context = canvas.getContext('2d');
    context.lineCap = 'round';
    context.lineJoin = 'round';
    context.fillStyle = '#ffffff';
    context.fillRect(0, 0, canvas.width, canvas.height);
    historyRef.current = [context.getImageData(0, 0, canvas.width,
canvas.height)];
  }
}, [gameState]);

const saveStateToHistory = () => {
  const canvas = canvasRef.current;
  const context = canvas.getContext('2d');
  historyRef.current.push(context.getImageData(0, 0, canvas.width,
canvas.height));
};

const undo = () => {
  if (historyRef.current.length > 1) {
    historyRef.current.pop();
    const previousState =
historyRef.current[historyRef.current.length - 1];
    const context = canvasRef.current.getContext('2d');
    context.putImageData(previousState, 0, 0);
  }
};

const startDrawing = ({ nativeEvent }) => {
  const { offsetX, offsetY } = nativeEvent;
  const context = canvasRef.current.getContext('2d');
  context.strokeStyle = isEraser ? '#ffffff' : currentColor;
  context.lineWidth = isEraser ? 20 : 5;
  context.beginPath();
  context.moveTo(offsetX, offsetY);
  setIsDrawing(true);
};

const draw = ({ nativeEvent }) => {
  if (!isDrawing) return;
  const { offsetX, offsetY } = nativeEvent;
  const context = canvasRef.current.getContext('2d');
  context.lineTo(offsetX, offsetY);
  context.stroke();
};

const stopDrawing = () => {

```

```

    if (isDrawing) {
      const context = canvasRef.current.getContext('2d');
      context.closePath();
      setIsDrawing(false);
      saveStateToHistory();
    }
  };

  const clearCanvas = () => {
    const canvas = canvasRef.current;
    const context = canvas.getContext('2d');
    context.fillStyle = 'ffffff';
    context.fillRect(0, 0, canvas.width, canvas.height);
    saveStateToHistory();
  };

  const finishDrawing = () => {
    const canvas = canvasRef.current;
    const base64Image = canvas.toDataURL('image/png');
    setFinalImage(base64Image);
    setGameState('result');
  };

  const startDrawPhase = (e) => {
    e.preventDefault();
    if (!phrase.trim()) return alert("Digite uma frase!");
    setTimeLimit(timeLimit);
    setGameState('draw');
  };

  const resetGame = () => {
    setPhrase('');
    setTimeLimit(60);
    setFinalImage(null);
    historyRef.current = [];
    setIsEraser(false);
    setGameState('write');
  };

  if (gameState === 'write') {
    return (
      <div style={styles.container}>
        <h2>Fase 1: Escolha a Frase</h2>
        <form onSubmit={startDrawPhase} style={styles.form}>
          <input type="text" placeholder="Digite o desafio..."
value={phrase} onChange={(e) => setPhrase(e.target.value)}
style={styles.input} />
          <div style={{ display: 'flex', gap: '10px' }}>
            <label>Tempo (segundos):</label>
            <input type="number" min="10" value={timeLimit}
onChange={(e) => setTimeLimit(Number(e.target.value))}
style={styles.input} />
          </div>
          <button type="submit"
style={styles.buttonPrimary}>Jogar</button>
        </form>
      </div>
    );
  }

  if (gameState === 'draw') {

```

```

    return (
      <div style={styles.container}>
        <h2>Fase 2: Desenhe!</h2>
        <p>Desenhe: <strong>{phrase}</strong> | ☐ {timeLeft}s</p>

        <div style={styles.toolbar}>
          {COLORS.map(color => (
            <button key={color} onClick={() => {
              setCurrentColor(color); setIsEraser(false); }}
              style={{...styles.colorBtn, backgroundColor: color}} />
          ))}
          <button onClick={() => setIsEraser(true)}>Borracha</button>
          <button onClick={undo}>Voltar</button>
          <button onClick={clearCanvas}>Limpar</button>
        </div>

        <canvas ref={canvasRef} width={700} height={400}
          style={styles.canvas} onMouseDown={startDrawing} onMouseMove={draw}
          onMouseUp={stopDrawing} onMouseLeave={stopDrawing} />
        <button onClick={finishDrawing}
          style={styles.buttonPrimary}>Terminei</button>
        </div>
      );
    }

    if (gameState === 'result') {
      return (
        <div style={styles.container}>
          <h2>Resultado Final</h2>
          <p>Frase: {phrase}</p>
          <img src={finalImage} alt="Desenho Final"
            style={styles.canvas} />
          <button onClick={resetGame} style={styles.buttonPrimary}>Jogar
            Novamente</button>
        </div>
      );
    }
  };

  const styles = {
    container: { display: 'flex', flexDirection: 'column', alignItems:
      'center', fontFamily: 'sans-serif', padding: '30px' },
    form: { display: 'flex', flexDirection: 'column', gap: '15px' },
    input: { padding: '10px', fontSize: '1rem', borderRadius: '4px',
      border: '1px solid #ccc' },
    toolbar: { display: 'flex', gap: '10px', marginBottom: '10px' },
    colorBtn: { width: '30px', height: '30px', borderRadius: '50%',
      cursor: 'pointer', border: 'none' },
    canvas: { border: '2px solid #333', borderRadius: '8px', cursor:
      'crosshair', backgroundColor: '#fff', marginTop: '10px', marginBottom:
      '10px' },
    buttonPrimary: { padding: '10px 20px', backgroundColor: '#3742fa',
      color: '#fff', border: 'none', borderRadius: '4px', cursor: 'pointer'
    }
  };

  const root = createRoot(document.getElementById("root"));
  root.render(<GarticClone />);

```